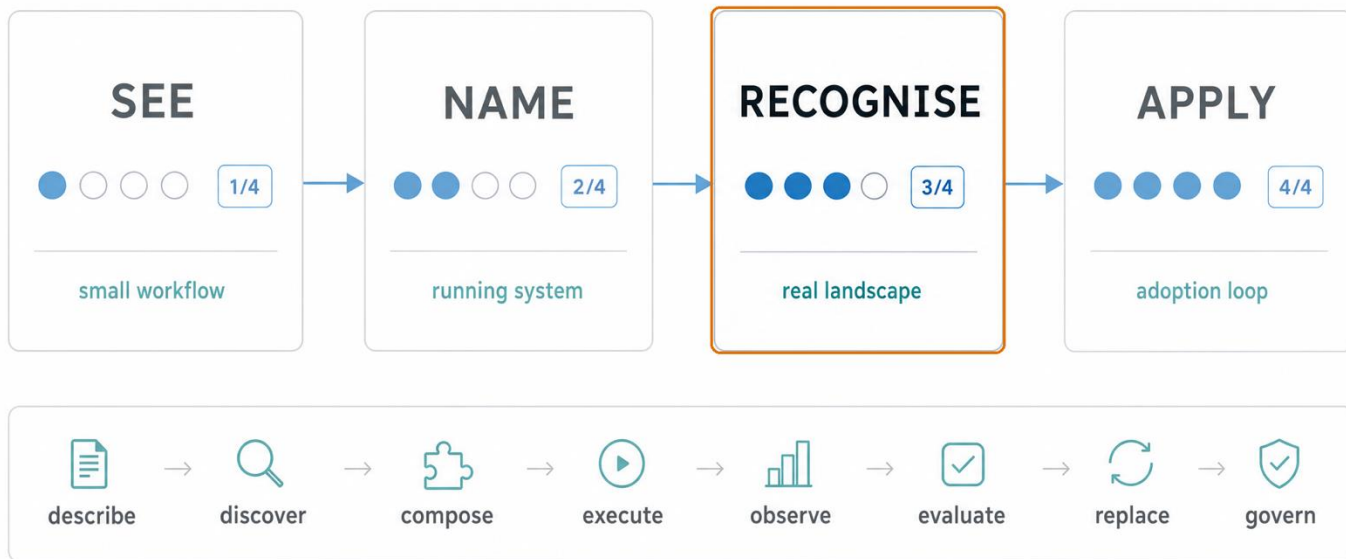


Course map: pass 3 of 4 — Recognise



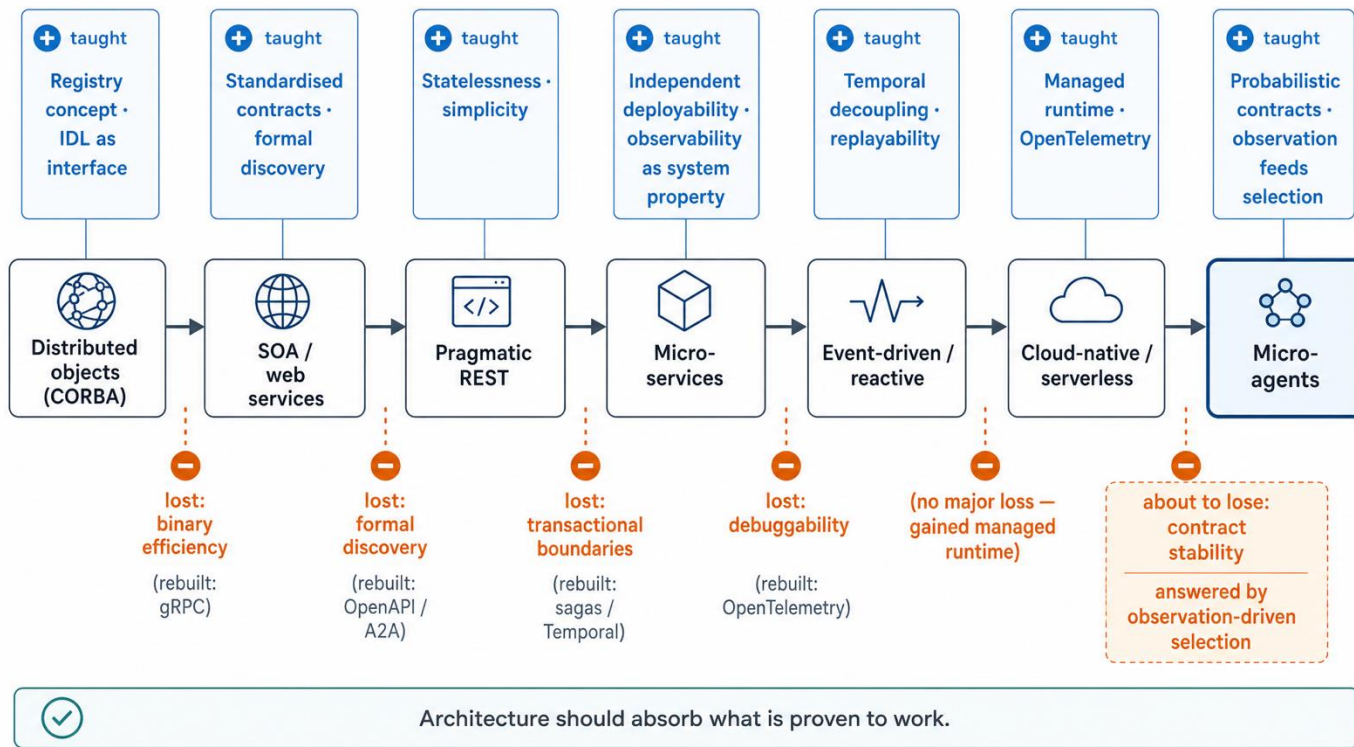
The Eras tour

 Era (years)	 Contract	 Discovery	 Coordination	 Observation	
 Distributed objects (1989)	IDL	OMG Trading Service	Activation Services	logs	
 SOA / web services (2000)	WSDL	UDDI	BPEL / ESB	logs / SLAs	
 Pragmatic REST (2000)	OpenAPI	(informal — lost)	application code	tracing	
 Microservices (2011)	OpenAPI / protobuf	Eureka / Consul / K8s	Airflow / Temporal	observability	
 Event-driven / reactive (2015)	Avro / Protobuf	Schema Registry	Kafka choreography	distributed tracing	
 Cloud-native / serverless (2018)	OpenAPI / gRPC	Service mesh	K8s + Argo	OpenTelemetry	
 Micro-agents (2024)	capability card / agent card	agent registry	planner / orchestrator	evals + traces	current unit: probabilistic capability



Same responsibilities; different unit behind the contract.

Long story short



UDDI is the registry warning label



registry can look correct
and still not be useful



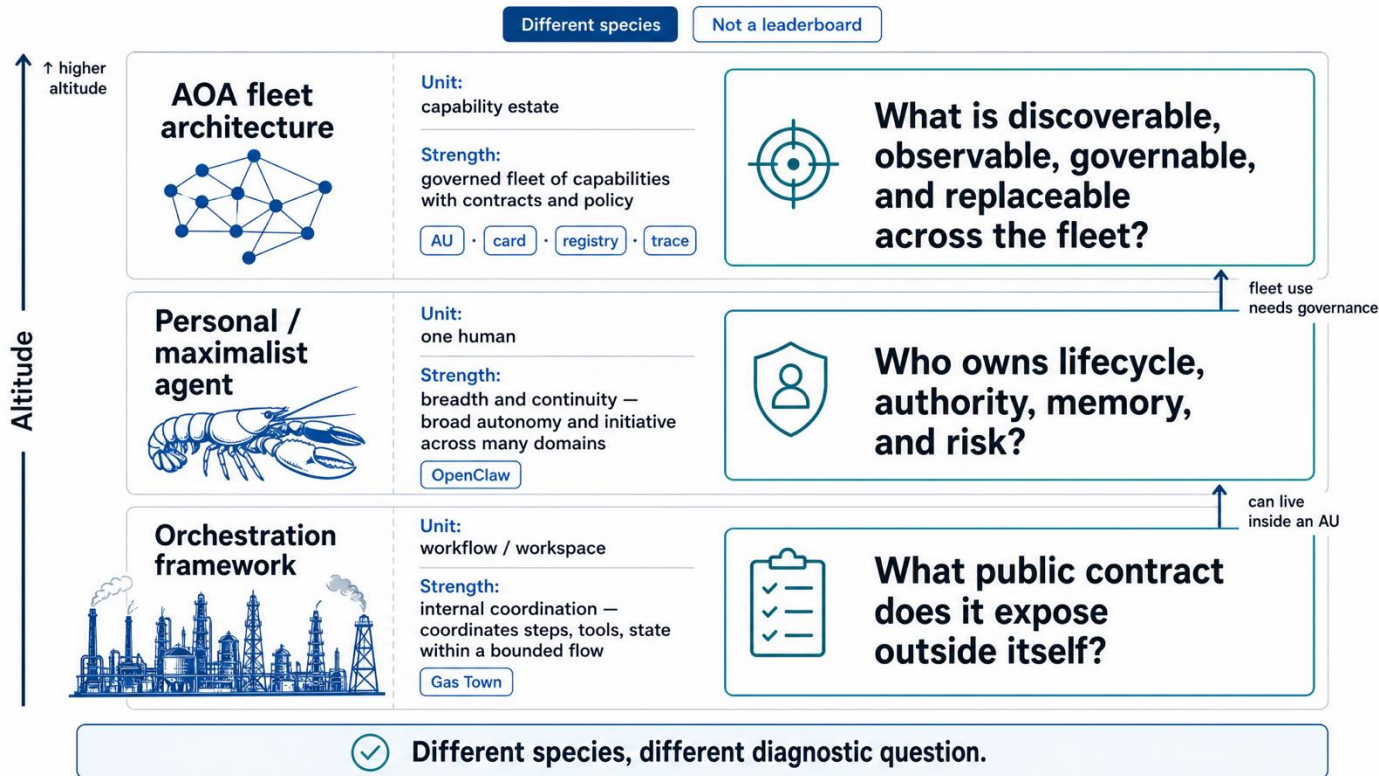
registry must be attached
to work and observation

 UDDI failure mode	design lesson	 AOA registry response
 publication friction	→	 lightweight agent cards
 static entries	→	 health and lifecycle signals
 no quality signal	→	 traces, evals, observed fitness
 weak consumption path	→	 planner consumes registry context
 stale phone book	→	 approve, discover, observe, deprecate

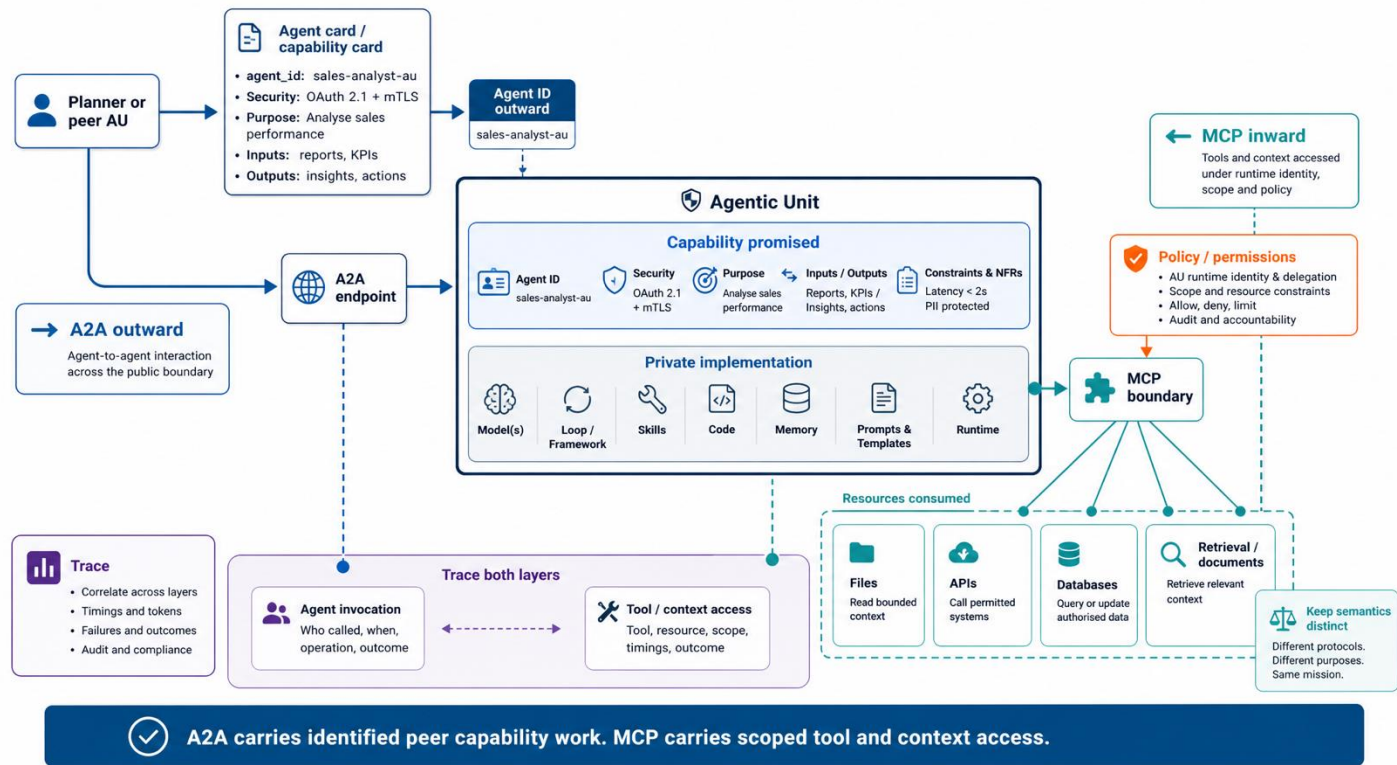


A registry must be used, measured, and governed — not merely published.

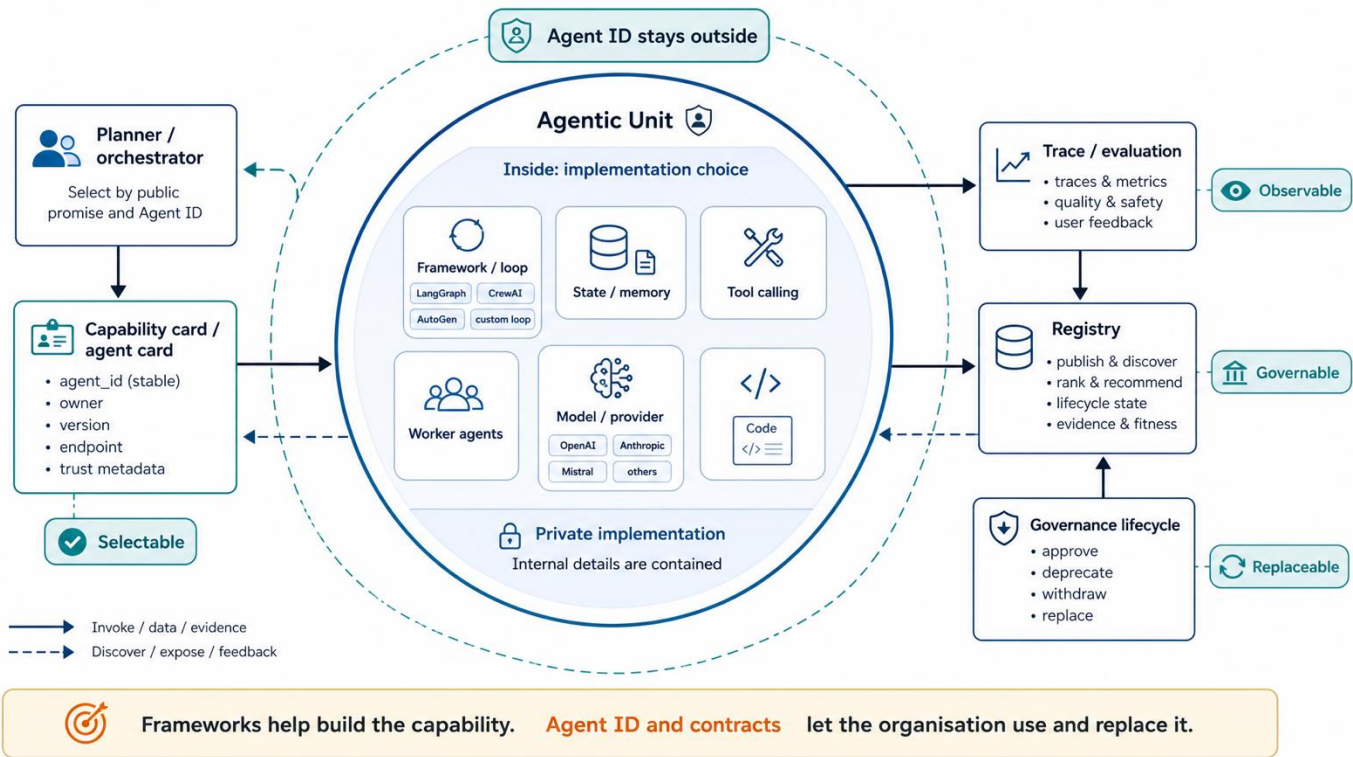
Three species, different altitude



A2A outside; MCP inside



Framework inside; contract outside



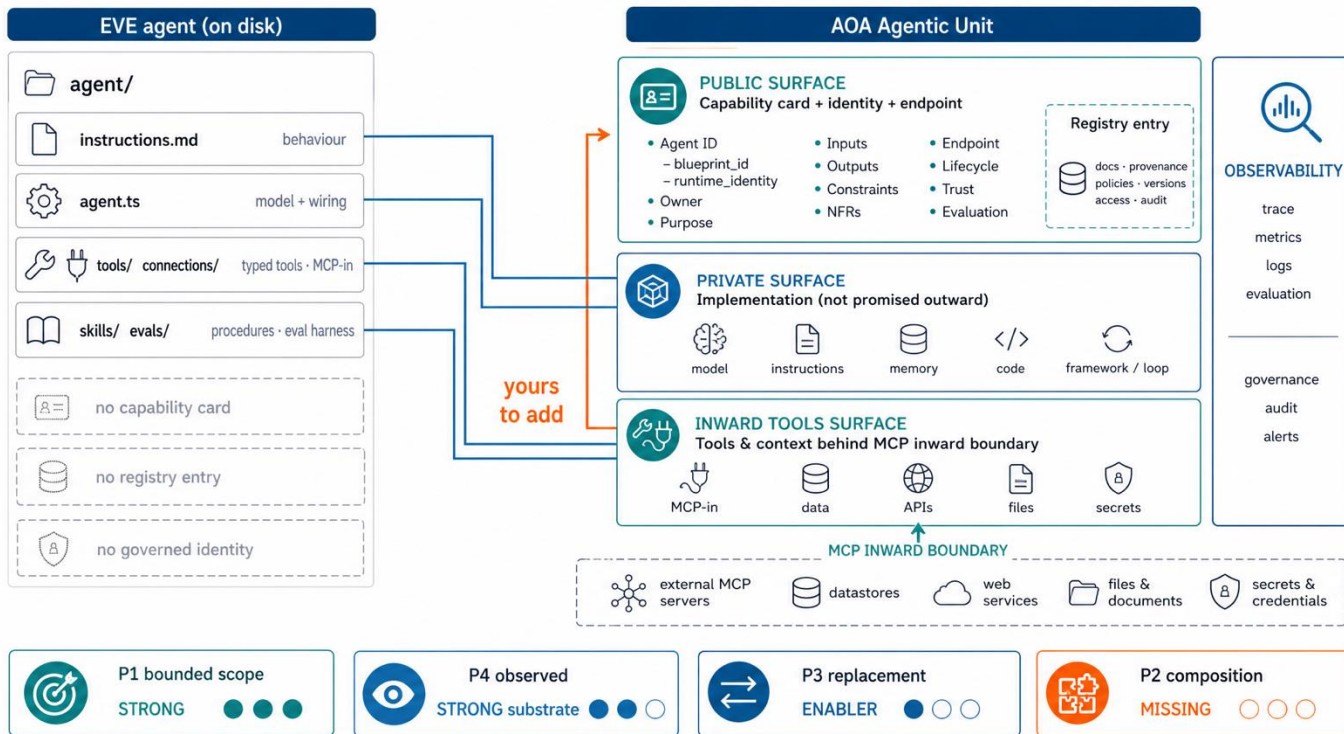
Platforms are pattern sources

	 contract	 discovery/register	 identity	 composition/ orchestration	 observation/ evaluation	 governance
which responsibilities does it discharge — and which remain yours to design?						
 AWS AgentCore		 dual-registry: A2A cards + MCP server.json				
 Google (Vertex / Gemini Enterprise + ARD)		 ARD: domain- anchored catalogs, June 2026	 SPIFFE per-agent IAM			 default-block gateway
 Vercel EVE new entrant	AU interior only					native runtime + outward AOA contract
 Microsoft (Foundry + Entra + Semantic Kernel)		 split across 3 surfaces	 Entra Agent Registry	 SK = orchestrator SDK		
 Salesforce  MuleSoft Salesforce + MuleSoft		 AgentExchange 1,000+		 fabric/broker		
 ServiceNow AI Control Tower						 AI Steward role
 solo.io  Camunda Solo.io Agentgateway / Camunda		 the 3 pieces A2A leaves out		 BPM: deterministic + audit	 the 3 pieces A2A leaves out	

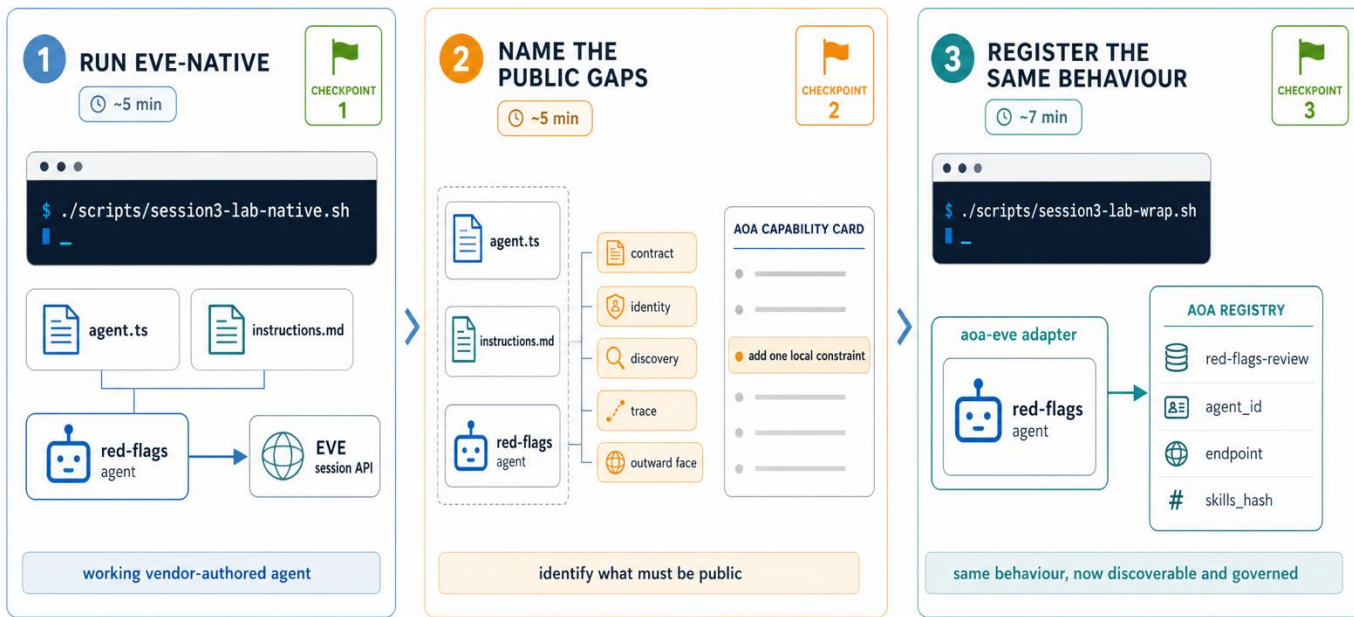


observation/evaluation as *selection input*: “Nobody ships observed-over-claimed. That one is yours.”

EVE inside; AOA outside



Lab: EVE native, then AOA contract



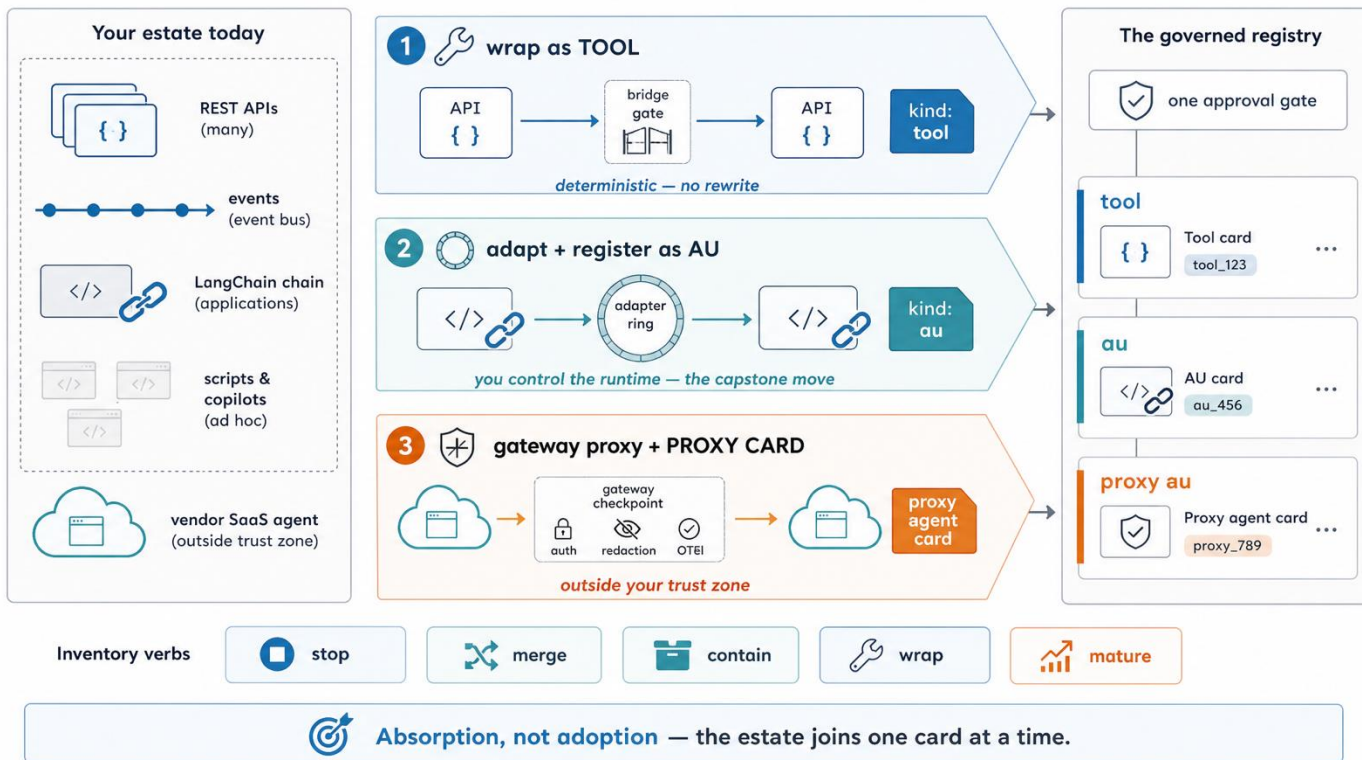
DRIVER



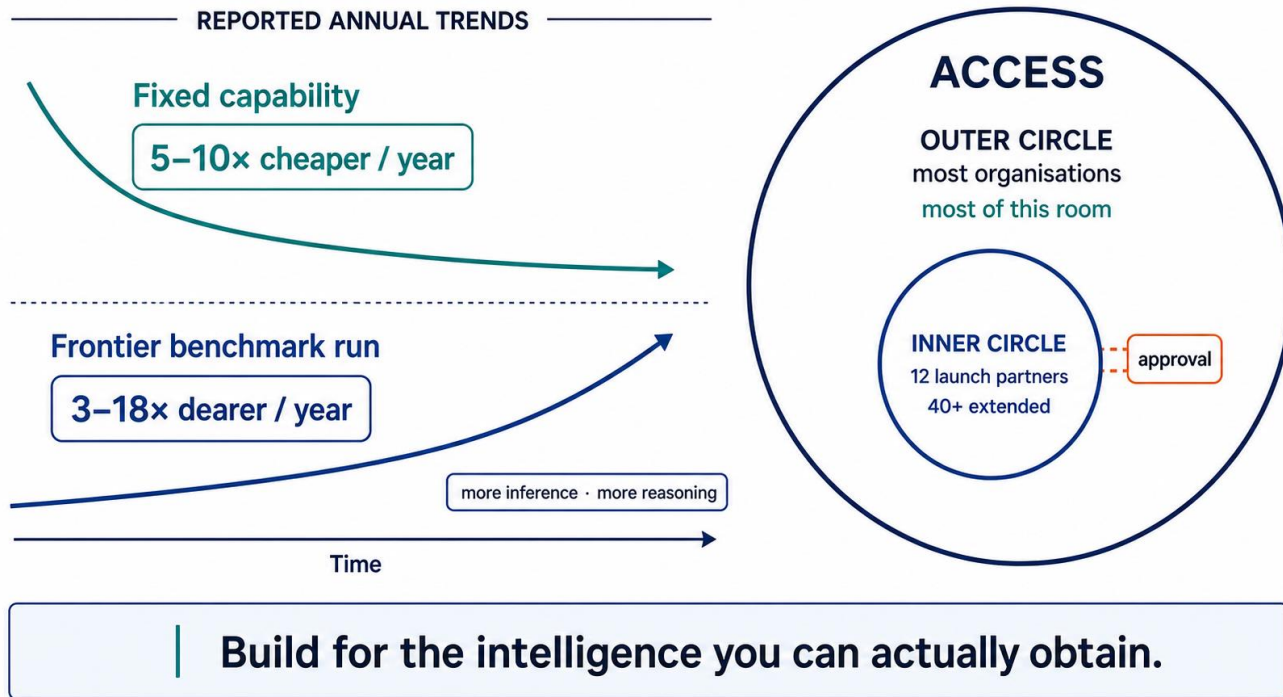
NAVIGATOR

Framework inside. Contract outside. Consumers do not inherit the framework choice.

You already have an estate

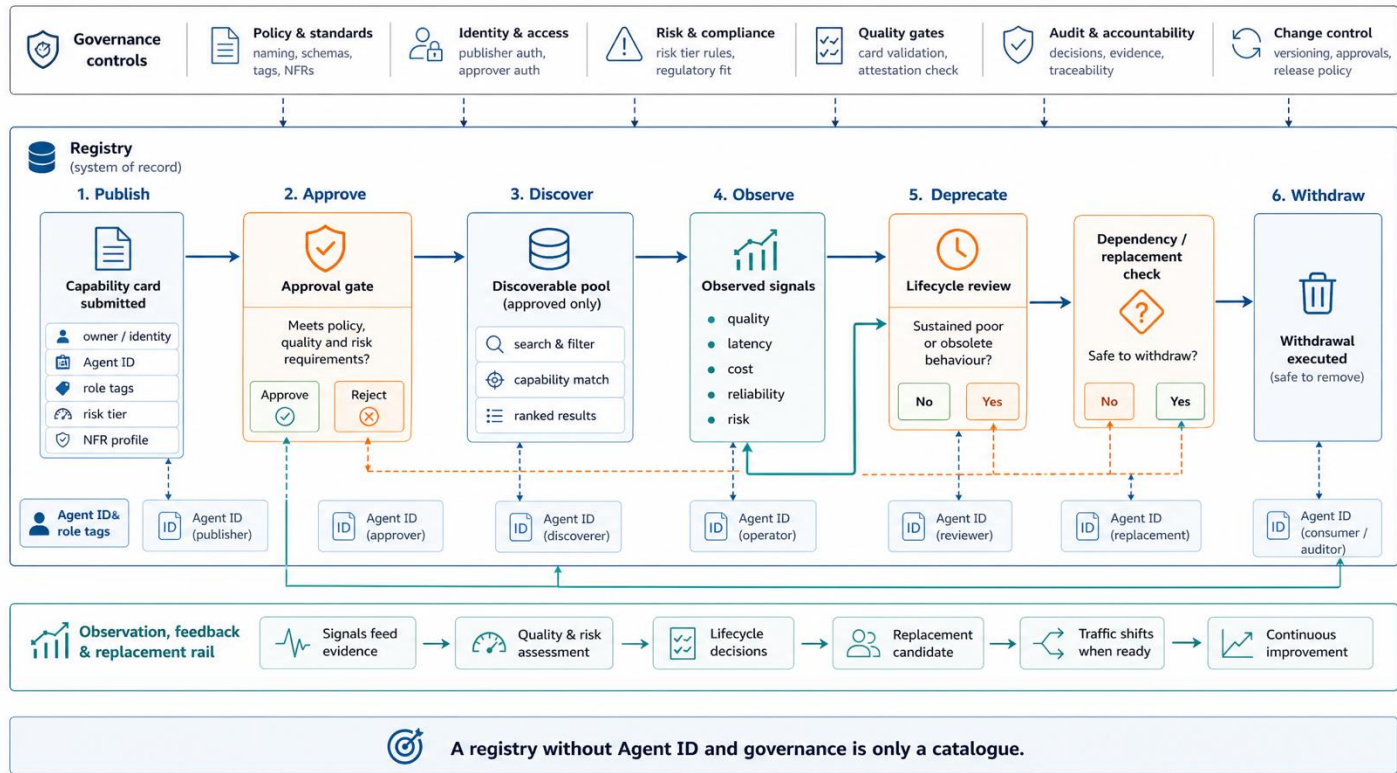


Tokenomics

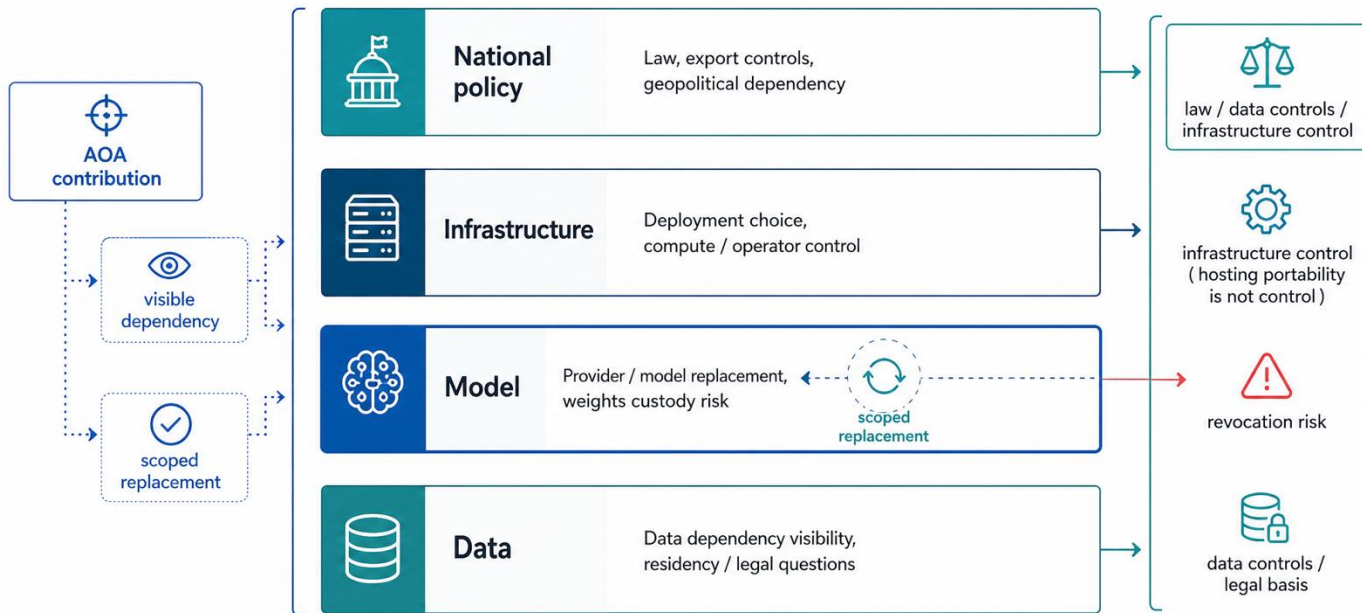


Sources: Gundlach et al., arXiv:2511.23455, Fig. 9 (2025/26); Anthropic, Project Glasswing (7 Apr 2026); OpenAI, Trusted Access for Cyber (7 May 2026).

Registry governance is lifecycle control



Sovereignty



AOA gives visibility and replacement paths; **sovereignty still has to be proven layer by layer.**

You can now recognise the shape

Question	What you now know	Evidence
1 Where is the boundary?	The framework fills the AU interior; the contract stays outside.	EVE native → registered AU
2 What is the public promise?	A2A exposes work; MCP exposes tools and context.	adapter + card + registry row
3 What would replacement take?	Contract and fallback paths must outlive one implementation.	registered alternative + lifecycle
4 What does the evidence say?	Cost, access and sovereignty are operating constraints.	lifecycle + scarcity + four-layer test
Handoff Next: apply the shape to your estate, evidence, and adoption boundary.		